UNIVERSIDADE DO ESTADO DO RIO GRANDE DO NORTE
DEPARTAMENTO DE INFORMÁTICA — CAMPUS CENTRAL
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

PAULO SÉRGIO SILVA DE MEDEIROS

ARGUS CYCLIST: UM ECOSSISTEMA MULTIPLATAFORMA E DE CÓDIGO ABERTO PARA O CICLISMO INDOOR

MOSSORÓ — RN
2026

PAULO SÉRGIO SILVA DE MEDEIROS

ARGUS CYCLIST: UM ECOSSISTEMA MULTIPLATAFORMA E DE CÓDIGO ABERTO PARA O CICLISMO INDOOR

Projeto de Trabalho de Conclusão de Curso apresentado ao Curso de Bacharelado em Ciência da Computação da Universidade do Estado do Rio Grande do Norte, como requisito parcial para obtenção do título de Bacharel em Ciência da Computação.

Orientador: Prof. Dr. Isaac de Lima Oliveira Filho
Professor da Disciplina: Prof. Me. Antonio Oliveira Filho

MOSSORÓ — RN
2026

RESUMO

O ciclismo virtual (*indoor cycling*) consolidou-se como modalidade esportiva reconhecida pela União Ciclística Internacional (UCI), porém o acesso a plataformas de simulação é restrito por assinaturas mensais em moeda estrangeira, exigência de hardware de alto desempenho e dependência contínua de conexão com a internet. O presente projeto apresenta o Argus Cyclist, um simulador multiplataforma de ciclismo virtual de código aberto sob licença GPLv3, desenvolvido em Go (Golang) com interface frontend em JavaScript por meio do framework Wails para desktop e Capacitor.js para dispositivos móveis Android. O sistema emprega arquitetura baseada em Clean Architecture com padrões Facade e Repository, abstraindo conexões Bluetooth Low Energy (BLE) para sensores de frequência cardíaca, potência e controle de rolos inteligentes via perfis GATT (`0x180D`, `0x1818`, `0x1826`). O motor de simulação física implementa a equação de Martin para cálculo de velocidade a partir de gradientes de rotas GPX, enquanto o módulo de análise fisiológica computa métricas avançadas como Potência Normalizada (NP), *Training Stress Score* (TSS), *Performance Management Chart* (PMC) e desvio cardiovascular (*Aerobic Decoupling*). Diferencia-se por incorporar um assistente de treino baseado em inteligência artificial generativa local (Ollama) para elaboração de planos personalizados no formato ZWO, além de um sistema de rastreamento de desgaste de componentes mecânicos da bicicleta. A validação experimental utiliza equipamentos reais de ciclismo para aferir latência de telemetria inferior a 150 ms e conformidade de arquivos `.fit` com o padrão Garmin. O projeto visa democratizar o treinamento científico de alto rendimento, eliminando barreiras econômicas e de conectividade.

Palavras-chave: Ciclismo virtual. Código aberto. Simulação física. Bluetooth Low Energy. Análise fisiológica. Inteligência artificial.

ABSTRACT

Virtual cycling has become a sport modality recognized by the Union Cycliste Internationale (UCI), yet access to simulation platforms is restricted by monthly subscription fees in foreign currency, high-performance hardware requirements, and continuous internet dependence. This project presents Argus Cyclist, an open-source cross-platform virtual cycling simulator licensed under GPLv3, developed in Go (Golang) with a JavaScript frontend using the Wails framework for desktop and Capacitor.js for Android mobile devices. The system employs a Clean Architecture with Facade and Repository patterns, abstracting Bluetooth Low Energy (BLE) connections for heart rate, power, and smart trainer control via GATT profiles (`0x180D`, `0x1818`, `0x1826`). The physics engine implements Martin's equation for speed calculation from GPX route gradients, while the physiological analysis module computes advanced metrics such as Normalized Power (NP), Training Stress Score (TSS), Performance Management Chart (PMC), and Aerobic Decoupling. It distinguishes itself by incorporating a local generative AI coaching assistant (Ollama) for creating personalized workout plans in ZWO format, along with a bicycle component wear tracking system. Experimental validation uses real cycling equipment to verify telemetry latency below 150 ms and `.fit` file compliance with the Garmin standard. The project aims to democratize scientific high-performance training by eliminating economic and connectivity barriers.

Keywords: Virtual cycling. Open source. Physics simulation. Bluetooth Low Energy. Physiological analysis. Artificial intelligence.

LISTA DE ILUSTRAÇÕES

Figura 1 – Arquitetura geral de componentes — Clean Architecture 18
Figura 2 – Fluxo sequencial de telemetria em tempo real 18
Figura 3 – Mapa mental do ecossistema Argus Cyclist 19

LISTA DE TABELAS

Tabela 1 – Comparativo entre plataformas de ciclismo virtual 8

SUMÁRIO

1 INTRODUÇÃO E A SITUAÇÃO ATUAL DO CICLISMO VIRTUAL 7
1.1 Referências a Outros Projetos e Estado da Arte 7

2 PROBLEMÁTICA 9

3 JUSTIFICATIVA 10
3.1 Relevância Tecnológica e Inovação Arquitetural 10
3.2 Relevância Científica (Modelagem Física e Fisiologia) 11
3.3 Relevância Social e Inclusão Esportiva 12

4 OBJETIVOS 13
4.1 Objetivo Geral 13
4.2 Objetivos Específicos 13

5 METODOLOGIA 15
5.1 Fases do Desenvolvimento 15

6 ARQUITETURA E DIAGRAMAS 17
6.1 Topologia de Componentes 17
6.2 Diagrama 1: Arquitetura Geral de Componentes (Clean Architecture) 17
6.3 Diagrama 2: Fluxo Sequencial de Telemetria de Baixa Latência . . 18
6.4 Diagrama 3: Mapa Mental do Ecossistema e Funcionalidades . . . 19

7 CONCLUSÃO 20

REFERÊNCIAS 21

1 INTRODUÇÃO E A SITUAÇÃO ATUAL DO CICLISMO VIRTUAL

O ciclismo virtual (*indoor cycling*) transitou de uma simples atividade alternativa para dias chuvosos para uma modalidade esportiva e científica de alta relevância global. Atualmente, é reconhecida pela União Ciclística Internacional (UCI) como uma modalidade oficial de *e-sports*, contando com campeonatos mundiais e atletas profissionais dedicados [4]. Este fenômeno foi impulsionado pelo surgimento de rolos de treino inteligentes (*smart trainers*), capazes de estabelecer comunicação bidirecional com computadores e dispositivos móveis para simular a resistência do terreno real em tempo real.

No entanto, a situação atual do ciclismo virtual é marcada por uma forte centralização de mercado e barreiras econômicas severas. Plataformas líderes como Zwift, Rouvy e TrainerRoad operam sob modelos de assinatura mensal proprietários com taxas cobradas em dólar comercial, entre $15 e $20 USD/mês. Para atletas em países em desenvolvimento, como o Brasil, o custo acumulado dessas assinaturas ultrapassa facilmente R$ 1.200,00 anuais, o que constitui um fator de exclusão para ciclistas amadores e atletas de base.

Além da barreira financeira, essas plataformas comerciais exigem hardware de alto desempenho com placas gráficas dedicadas, inviabilizando o uso por atletas que possuem apenas computadores de entrada ou dispositivos móveis comuns. Adicionalmente, tais soluções dependem de conexão contínua com a internet para computar métricas, o que impede treinos em ambientes *offline* ou com conectividade instável.

O Argus Cyclist propõe-se a ser uma alternativa nacional, multiplataforma, *offline-first* e de código aberto, unindo simulação física interativa, telemetria de precisão, assistência de inteligência artificial generativa local e rastreamento de desgaste de componentes, sem taxas recorrentes e com baixíssimos requisitos de hardware.

1.1 REFERÊNCIAS A OUTROS PROJETOS E ESTADO DA ARTE

Para contextualizar a relevância e o posicionamento do Argus Cyclist, apresenta-se a seguir um comparativo com as principais plataformas do mercado e iniciativas de código aberto:

- **Zwift**: O líder absoluto do mercado foca em um ambiente virtual tridimensional massivo *online*, com forte apelo social e gamificação. Suas limitações residem na ausência de modo *offline*, custo recorrente elevado e exigência de hardware potente para processamento gráfico 3D.

- **Rouvy**: Diferencia-se por simular rotas reais por meio de vídeos gravados de estradas ao redor do mundo. Apresenta alta demanda de largura de banda de rede para transmissão de vídeo e sofre dos mesmos problemas de custo de assinatura fechada.

- **TrainerRoad**: Foca exclusivamente em treinos estruturados baseados em faixas de potência (telemetria pura), sem simulação gráfica complexa. Suas métricas e planos de treino são proprietários e bloqueados por assinaturas de custo elevado.

- **GoldenCheetah**: Software livre sob licença GPLv3 voltado à análise científica pós-treino. Embora possua ferramentas matemáticas excelentes (como o gráfico PMC e modelos de fadiga), sua interface é extremamente complexa, com uma curva de aprendizado íngreme, e ele não foi projetado para atuar como um simulador de rotas interativo e responsivo em tempo real com controle de rolos inteligentes.

- **Argus Cyclist**: Posiciona-se no estado da arte ao preencher a lacuna entre a análise de dados científica (típica do GoldenCheetah) e a simulação de rotas interativa e gamificada (típica do Zwift), operando de forma totalmente gratuita, multiplataforma, *offline-first* e com processamento local eficiente. Diferencia-se ainda por oferecer um treinador virtual baseado em IA generativa local (Ollama) [8] para elaboração de treinos personalizados em linguagem natural e um sistema de rastreamento de desgaste de componentes que calcula automaticamente a vida útil restante de cada peça da bicicleta com base na distância percorrida.

Tabela 1 – Comparativo entre plataformas de ciclismo virtual

Plataforma	**Licença**	**Execução**	**Hardware**	**Offline**	**Foco Principal**
Zwift	Proprietária ($20/mês)	Nuvem	Alto	Não	Gamificação e Social
Rouvy	Proprietária (€15/mês)	Nuvem	Médio-Alto	Não	Realismo de rotas reais
TrainerRoad	Proprietária ($22/mês)	Nuvem	Baixo	Não	Treinos Estruturados
GoldenCheetah	Livre (GPLv3)	Local	Baixo	Sim	Análise Científica pós-treino
Argus Cyclist	**Livre (GPLv3)**	**Local (Edge)**	**Baixo**	**Sim**	**Física + Telemetria + IA**

2 PROBLEMÁTICA

O desenvolvimento do projeto Argus Cyclist é motivado por quatro problemas centrais identificados no ecossistema atual do ciclismo virtual:

1. **Exclusão Socioeconômica e Barreiras de Acesso**: O modelo de monetização em moeda estrangeira das plataformas de *e-sports* de ciclismo inviabiliza o treinamento científico e estruturado para atletas de base e ciclistas amadores de menor poder aquisitivo no Brasil.

2. **Dependência Crônica de Conectividade (*Cloud Lock-in*)**: A arquitetura das soluções comerciais centraliza a lógica do simulador e o processamento de métricas em servidores na nuvem. Quedas temporárias de conexão resultam no encerramento abrupto das atividades ou na perda de treinos inteiros, além de inviabilizar o uso em locais sem sinal de internet.

3. **Vulnerabilidade de Privacidade de Dados**: A coleta de dados biométricos (frequência cardíaca contínua) e geográficos (coordenadas GPS de rotas importadas) é armazenada de forma compulsória em servidores privados estrangeiros, infringindo princípios de soberania de dados e privacidade por padrão (*Privacy-by-Design*).

4. **Monetização Fisiológica Abusiva**: Métricas avançadas de controle de carga de treinos fundamentais para evitar o *overtraining* e prescrever a recuperação (como PMC, TSS e CTL) dependem apenas de equações matemáticas consolidadas na literatura científica de fisiologia do esporte. Ainda assim, são vendidas como recursos exclusivos em softwares proprietários pagos.

3 JUSTIFICATIVA

O projeto Argus Cyclist justifica-se pela sua contribuição tecnológica, científica e de impacto social no cenário esportivo e de engenharia de software:

3.1 RELEVÂNCIA TECNOLÓGICA E INOVAÇÃO ARQUITETURAL

Do ponto de vista da Engenharia de Software, o projeto aborda o desafio de criar um sistema multiplataforma de tempo real de baixíssima latência. A escolha do Go (Golang) no *backend* deve-se à sua capacidade nativa de concorrência baseada em *Goroutines* e *Channels*, permitindo realizar escaneamento e manter conexões estáveis com múltiplos sensores Bluetooth Low Energy (BLE) utilizando a biblioteca TinyGo Bluetooth [11] usando perfis GATT específicos (frequência cardíaca [1], potência e controle bidirecional FTMS) sem congelar a interface de usuário.

A integração com o framework Wails [12] viabiliza um binário de desktop extremamente leve, utilizando a *web engine* nativa do sistema operacional para renderizar uma interface reativa em JavaScript (Vite), registrando um consumo de memória inferior a 200 MB uma fração do demandado por soluções baseadas em Electron. Para dispositivos móveis, a utilização do Capacitor.js [13] possibilita reutilizar a mesma base de código do *frontend* para compilar um aplicativo nativo para Android, rodando um motor físico portado inteiramente para JavaScript para simular e controlar os rolos diretamente de um celular comum via antenas BLE integradas.

Inovadoramente, o sistema incorpora um módulo de IA generativa local que se comunica com uma instância do Ollama [8] rodando no próprio dispositivo do usuário. Esta arquitetura elimina completamente a dependência de serviços em nuvem para funcionalidades de *coaching*, garantindo que nenhum dado biométrico ou de treino seja transmitido a servidores externos. A escolha de modelos pequenos (três bilhões de parâmetros, como o Qwen 2.5:3B) viabiliza a execução mesmo em hardware modesto, e o tempo de resposta estendido (300 s) acomoda a inferência em CPU.

O sistema de controle de desgaste de componentes implementa um modelo de depreciação linear simples, porém eficaz, onde cada componente cadastrado (corrente, cassete, coroas, etc.) tem sua vida útil definida em quilômetros pelo usuário, e o desgaste percentual é atualizado automaticamente ao final de cada sessão de treino com base na distância percorrida. Este sistema segue uma arquitetura em camadas bem definida com persistência em SQLite [9] local, mantendo a coerência com o princípio *offline-first* do projeto.

3.2 RELEVÂNCIA CIENTÍFICA (MODELAGEM FÍSICA E FISIOLOGIA)

O sistema aplica modelagens matemáticas rigorosas de física e fisiologia do esporte diretamente no dispositivo do usuário:

Dinâmica de Movimento (Equação de Martin [3]): O motor físico calcula a aceleração e a velocidade virtual aplicando a conservação de energia sobre as forças resistivas. A equação fundamental que relaciona a potência do ciclista (P) com a velocidade (v) é dada por:

$$P = v \cdot \left(\frac{1}{2} \rho C_d A \, v^2 + mgC_{rr} + mg \sin\theta + ma \right) \cdot \eta^{-1} \tag{3.1}$$

onde:

- ρ é a densidade do ar ($1.225\,\mathrm{kg\,m^{-3}}$ ao nível do mar);
- C_dA é o coeficiente de arrasto aerodinâmico vezes a área frontal;
- m é a massa total (ciclista + bicicleta);
- g é a aceleração gravitacional ($9.81\,\mathrm{m\,s^{-2}}$);
- C_{rr} é o coeficiente de resistência ao rolamento;
- θ é o ângulo de inclinação da via (derivado do gradiente percentual da rota GPX);
- a é a aceleração linear;
- η é a eficiência da transmissão mecânica.

Fisiologia e Carga de Treino: Processa o vetor de potência de 1 Hz para extrair as seguintes métricas:

A Potência Normalizada (NP) é calculada pela média móvel de 30 segundos elevada à quarta potência:

$$NP = \sqrt[4]{\frac{1}{N} \sum_{i=1}^{N} p_{30,i}^4} \tag{3.2}$$

onde $p_{30,i}$ é a i-ésima média móvel de 30 segundos do vetor de potência.

O Fator de Intensidade (IF) e o *Training Stress Score* (TSS) são dados por:

$$IF = \frac{NP}{FTP} \tag{3.3}$$

$$TSS = \frac{d \cdot NP \cdot IF}{FTP \cdot 3600} \times 100 \tag{3.4}$$

onde d é a duração do treino em segundos e FTP é a potência funcional limiar (*Functional Threshold Power*).

O *Performance Management Chart* (PMC) é compilado a partir das três métricas de carga:

$$CTL_n = CTL_{n-1} + \frac{TSS_n - CTL_{n-1}}{42} \tag{3.5}$$

$$ATL_n = ATL_{n-1} + \frac{TSS_n - ATL_{n-1}}{7} \tag{3.6}$$

$$TSB = CTL - ATL \tag{3.7}$$

onde CTL (*Chronic Training Load*) representa o nível de condicionamento físico (*Fitness*), ATL (*Acute Training Load*) representa a fadiga acumulada, e TSB (*Training Stress Balance*) representa a forma atual do atleta.

Adicionalmente, computa o Desvio Cardiovascular (*Aerobic Decoupling* — Pw:HR) comparando a relação potência-frequência cardíaca na primeira e segunda metade de treinos de ritmo constante:

$$Decoupling = \left(\frac{P_{\text{segunda metade}}}{HR_{\text{segunda metade}}} \div \frac{P_{\text{primeira metade}}}{HR_{\text{primeira metade}}} - 1 \right) \times 100\% \tag{3.8}$$

valores superiores a 5% alertam o atleta sobre desidratação ou fadiga cardíaca excessiva.

3.3 RELEVÂNCIA SOCIAL E INCLUSÃO ESPORTIVA

Ao prover uma solução *offline* de código aberto e totalmente gratuita compatível com computadores modestos e celulares Android, o Argus Cyclist atua como um elemento de inclusão social. O projeto permite que ciclistas locais e equipes amadoras de ciclismo utilizem equipamentos de rolo de treino inteligente sem taxas mensais, democratizando o treinamento esportivo de alto rendimento.

4 OBJETIVOS

4.1 OBJETIVO GERAL

Projetar, desenvolver e validar o Argus Cyclist, um simulador multiplataforma de ciclismo virtual de código aberto e gratuito, integrado a um sistema científico de análise de telemetria esportiva *offline-first*, garantindo processamento em borda (*Edge*) de baixa latência e interoperabilidade com hardware padrão do mercado.

4.2 OBJETIVOS ESPECÍFICOS

1. **Conectividade de Hardware**: Implementar rotinas robustas em Go para escaneamento e conexão com periféricos via Bluetooth Low Energy (BLE), decodificando serviços GATT para cintas cardíacas (`0x180D`), sensores de potência (`0x1818`) e rolos de treino controlados (FTMS — `0x1826`) [1].

2. **Motor de Simulação Física**: Desenvolver algoritmos que traduzam o gradiente das coordenadas de uma rota GPX [6] em resistência de torque mecânico no rolo de treino inteligente, atualizando dinamicamente a resistência conforme o peso do ciclista e sua inclinação virtual, conforme a Equação 3.1.

3. **Interface Gráfica de Alta Performance**: Construir um HUD (*Heads-Up Display*) leve de 60 fps usando Canvas/WebGL, integrando o MapLibre GL JS [7] com renderização topográfica em 3D Terrain-RGB, o ECharts [5] para visualização de gráficos de telemetria e rotas dinâmicas coloridas de acordo com a inclinação do terreno.

4. **Análise Fisiológica Local**: Implementar funções matemáticas para cálculo contínuo de métricas esportivas avançadas (NP, IF, TSS, TRIMP) e visualização do gráfico de evolução histórica de carreira PMC (*Performance Management Chart*), conforme as Equações 3.2 a 3.7.

5. **Assistente de Treino por IA Generativa Local (IA Coach)**: Integrar um assistente conversacional de treinamento esportivo utilizando modelos de linguagem de pequena escala (3B parâmetros) executados localmente via Ollama [8], capaz de interpretar o perfil do atleta e gerar planos de treino estruturados no formato ZWO (*Zwift Workout*) com carregamento automático no modo ERG do simulador.

6. **Controle de Desgaste de Componentes Mecânicos**: Implementar um sistema de cadastro e monitoramento de componentes da bicicleta com cálculo automático de

desgaste percentual baseado na quilometragem acumulada, histórico de substituições com auditoria temporal e indicadores visuais codificados por cores (verde, amarelo, vermelho) para alertar o atleta sobre a necessidade de manutenção preventiva.

7. **Persistência de Dados Confiável**: Modelar um banco de dados local com SQLite [9] usando o padrão *Repository* e *Facade* para persistir usuários, atividades e recordes pessoais de forma 100% *offline* e estruturada.

8. **Interoperabilidade de Dados**: Desenvolver rotinas para compilação e exportação de dados na estrutura de arquivo binário padrão `.FIT` (Garmin/ANT+ SDK) [2] e integração de upload assíncrono via API do Strava [10].

9. **Portabilidade Multiplataforma**: Compilar a aplicação para sistemas Android, Windows, macOS e Linux utilizando tecnologias multiplataforma, adaptando o motor físico para JavaScript a fim de garantir operação autônoma, compatibilidade entre diferentes ambientes e baixa latência em dispositivos móveis e *desktops*.

10. **Gamificação Saudável**: Integrar um sistema de níveis e conquistas cardiovasculares persistentes com uma *Trophy Room* interativa 3D/Canvas para engajamento dos atletas amadores.

5 METODOLOGIA

O trabalho adota uma metodologia de pesquisa aplicada, orientada ao desenvolvimento tecnológico experimental, utilizando abordagens qualitativas e quantitativas para validação dos resultados. O processo de engenharia de software baseia-se em um modelo de desenvolvimento iterativo e incremental (práticas ágeis), dividido em quatro fases essenciais:

5.1 FASES DO DESENVOLVIMENTO

1. **Fase de Levantamento e Especificação**: Estudo aprofundado dos perfis GATT Bluetooth SIG (*Heart Rate Service*, *Cycling Power Service* e *Fitness Machine Service*) e decodificação hexadecimal de *frames* de telemetria. Análise da documentação do Garmin FIT SDK [2] para modelar o codificador de arquivos binários e das APIs REST do Strava sob protocolo de segurança OAuth 2.0 [10].

2. **Fase de Definição Arquitetural**: Desenho do padrão estrutural do sistema sob conceitos de *Clean Architecture*. Separação rígida de responsabilidades em camadas (Apresentação, Casos de Uso, Serviços de Domínio e Persistência de Dados). Seleção da *stack* multiplataforma ideal: *backend* em Go (Golang) com *bindings* Wails (Desktop) e empacotamento móvel nativo em Android via Capacitor.js [13].

3. **Fase de Implementação e Refatoração**: Implementação dos serviços em Go: módulo de telemetria (`internal/service/ble`) com *goroutines* concorrentes de escuta; módulo físico (`internal/service/sim`) com suporte a GPX e testes de FTP; persistência em banco local (`internal/repository/sqlite`) sob padrão Facade (`usecase/storage_facade`); e exportador esportivo (`internal/service/fit`). No *frontend* JavaScript, implementa-se o `TelemetryEventBus` para desacoplar a atualização de tela da chegada de dados BLE, garantindo renderização fluida a 60 fps.

4. **Fase de Testes e Validação Experimental**: Execução de testes funcionais e de integração utilizando equipamentos reais de ciclismo próprios, conectando o Argus Cyclist a rolos inteligentes (*smart trainers*) e medidores de potência para validação prática do sistema em ambiente de uso real. A validação quantitativa consiste na medição da latência de telemetria (requisitada para operar em $< 150\,\text{ms}$ do hardware até a tela) e em testes de conformidade estrutural, submetendo os arquivos binários `.fit` gerados diretamente ao Strava [10], atestando a integridade estatística dos dados em relação ao padrão global da indústria.

Dessa forma, a metodologia proposta garante a consistência científica no desenvolvimento e assegura a qualidade física da simulação interativa, resultando em um produto de software estável e utilizável por ciclistas.

6 ARQUITETURA E DIAGRAMAS

A arquitetura do Argus Cyclist foi desenhada para garantir o máximo isolamento da lógica de negócio em relação ao hardware e às interfaces visuais, assegurando que o simulador funcione com alto desempenho tanto no computador quanto no celular.

6.1 TOPOLOGIA DE COMPONENTES

A arquitetura do Argus Cyclist segue princípios de *Clean Architecture* e padrões *Facade* e *Repository*. O *backend* em Go abstrai as conexões e os cálculos de física clássica, exportando métodos assíncronos para a camada de apresentação em JavaScript através de IPC (*bindings* Wails). No *mobile*, o Capacitor encapsula a aplicação web e roda um motor físico em JavaScript. A divisão lógica do sistema estrutura-se nas seguintes camadas principais:

1. **Camada de Apresentação (Frontend Vanilla JS)**: Gerencia a interface de usuário, gráficos Canvas e o barramento de eventos `TelemetryEventBus` que atualiza a tela de forma assíncrona.

2. **Wails IPC Bridge (Desktop)**: Camada de comunicação interprocesso que conecta a lógica do *frontend* ao núcleo do *backend*.

3. **Camada de Aplicação (Controladores/Casos de Uso Go)**: Estrutura a fachada `StorageFacade` e os controladores de treinos (`WorkoutController`) e desafios (`ChallengeController`).

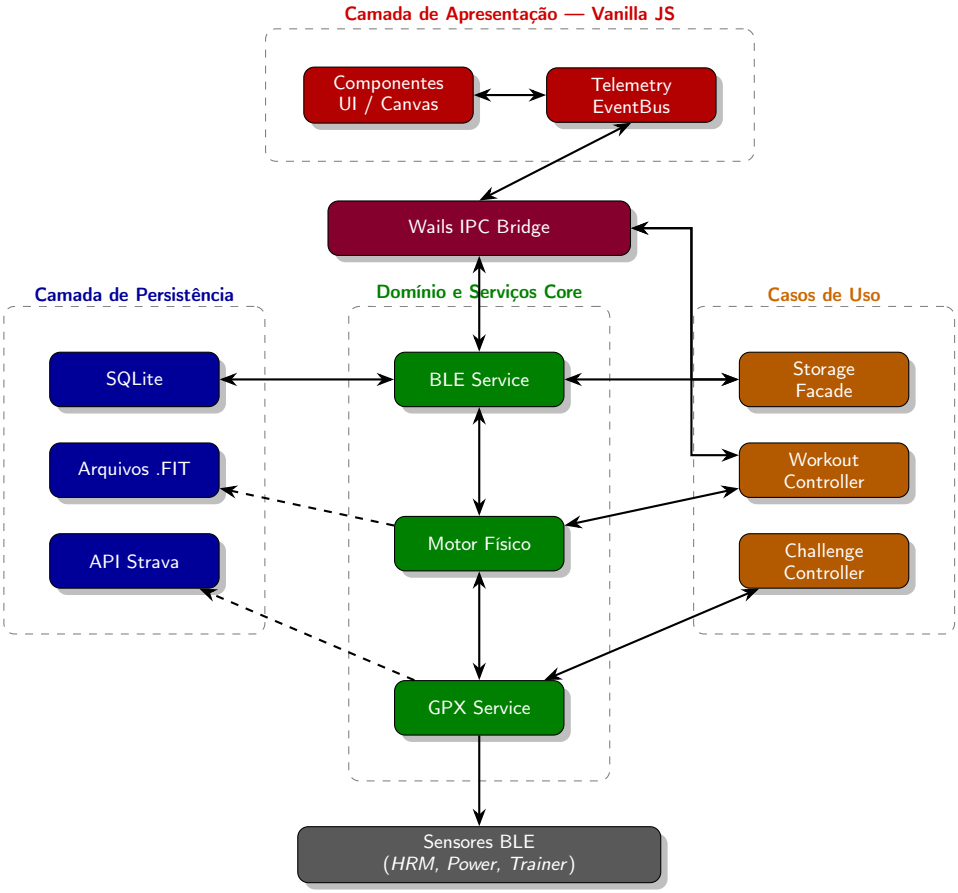
4. **Camada de Domínio / Serviços Core (Go)**: Compreende o motor físico de simulação baseada em dados GPX e o gerenciador de conexões Bluetooth LE (assinaturas GATT).

5. **Camada de Persistência (SQLite / FIT / Strava)**: Mapeia os repositórios para o banco SQLite local e coordena a geração do arquivo `.FIT` e a transmissão segura à API do Strava.

6.2 DIAGRAMA 1: ARQUITETURA GERAL DE COMPONENTES (CLEAN ARCHITECTURE)

O diagrama da Figura 1 ilustra a interação entre as camadas do sistema, destacando a separação de responsabilidades e o fluxo de comunicação entre o Frontend (JavaScript) e o Backend (Go).

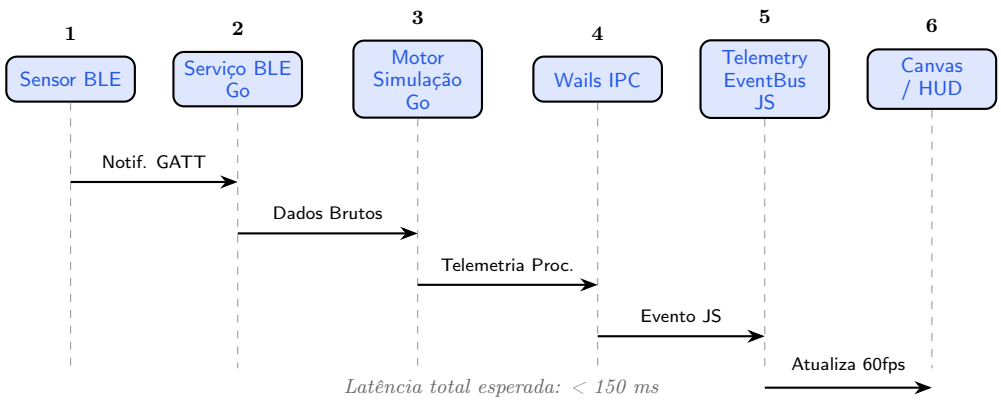
Figura 1 – Arquitetura geral de componentes — Clean Architecture



6.3 DIAGRAMA 2: FLUXO SEQUENCIAL DE TELEMETRIA DE BAIXA LATÊNCIA

A simulação de ciclismo virtual exige uma comunicação de baixíssima latência. O diagrama de sequência da Figura 2 demonstra como os dados trafegam do sensor até a renderização gráfica no momento do pedal.

Figura 2 – Fluxo sequencial de telemetria em tempo real



6.4 DIAGRAMA 3: MAPA MENTAL DO ECOSSISTEMA E FUNCIONALIDADES

A Figura 3 apresenta o mapa mental do ecossistema Argus Cyclist, organizando as principais funcionalidades e suas inter-relações de forma totalmente legível.

Figura 3 – Mapa mental do ecossistema Argus Cyclist



7 CONCLUSÃO

O desenvolvimento do Argus Cyclist demonstra a viabilidade técnica de uma plataforma de ciclismo virtual de código aberto, multiplataforma e orientada ao processamento local. O projeto diferencia-se por combinar simulação física em tempo real, análise fisiológica avançada, integração com dispositivos BLE e recursos de inteligência artificial executados localmente, preservando a privacidade dos usuários e reduzindo dependências de infraestrutura em nuvem. Adicionalmente, o sistema de rastreamento de desgaste de componentes oferece uma ferramenta prática de manutenção preventiva, preenchendo uma lacuna não endereçada pelas plataformas comerciais existentes.

Além da contribuição tecnológica, a proposta possui relevância social significativa ao democratizar o acesso ao treinamento esportivo científico, especialmente para atletas amadores e usuários com limitações financeiras ou computacionais. Dessa forma, o Argus Cyclist consolida-se como uma alternativa inovadora às soluções proprietárias do mercado, unindo acessibilidade, desempenho e liberdade tecnológica.

REFERÊNCIAS

[1] BLUETOOTH SIG. **Fitness Machine Service (FTMS) Specification v1.0**. Bluetooth Special Interest Group, 2017. Disponível em: https://www.bluetooth.com/specifications/specs/.

[2] GARMIN. **FIT SDK (Flexible and Interoperable Data Transfer)**. ANT+ Alliance, Garmin Canada, 2024. Disponível em: https://developer.garmin.com/fit/overview/.

[3] MARTIN, J. C. *et al.* Validation of a Mathematical Model for Road Cycling Power. **Journal of Applied Biomechanics**, Human Kinetics, v. 14, n. 3, p. 276–291, 1998.

[4] COGGAN, A. **Training and Racing with a Power Meter**. 2. ed. Boulder, CO: VeloPress, 2010.

[5] ECHARTS. **Apache ECharts Documentation v5**. Apache Software Foundation, 2024. Disponível em: https://echarts.apache.org/.

[6] TOPOGRAFIX. **GPX: GPS Exchange Format v1.1**. 2004. Disponível em: https://www.topografix.com/gpx.asp.

[7] MAPLIBRE. **MapLibre GL JS Documentation v4**. MapLibre Community, 2024. Disponível em: https://maplibre.org/.

[8] OLLAMA. **Ollama: Local LLM Inference Platform**. 2024. Disponível em: https://ollama.ai/.

[9] SQLITE CONSORTIUM. **SQLite Documentation v3**. 2024. Disponível em: https://sqlite.org/.

[10] STRAVA API. **Strava Developers API Reference v3**. Strava Inc., 2025. Disponível em: https://developers.strava.com/.

[11] TINYGO. **TinyGo Bluetooth: BLE Library for Go**. 2024. Disponível em: https://tinygo.org/x/bluetooth/.

[12] WAILS. **Wails: Create Beautiful Desktop Applications using Go and HTML/CSS/JS v2**. Wails Project, 2024. Disponível em: https://wails.io/.

[13] CAPACITOR. **Capacitor: Cross-platform Native Runtime for Web Apps v5**. Ionic Project, 2024. Disponível em: https://capacitorjs.com/.